COLLABRIX

A Team-First Freelance Collaboration Platform

A Project Report

Submitted by

Jaghatha Uday Prakash
2303031240447

Boddu Venkata Sai Kowshik
2303031241591

Katreddi Hima Nagendra
2303031240520

Immadi Venkata Rama Chaitanya Vardhan
2303031240436

In Partial Fulfilment for the
Award of the Degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING (Spl. In Artificial Intelligence)

Under the Guidance of

Prof. (Dr.) Barkha Makwana

PARUL UNIVERSITY, VADODARA

October – 2026

PARUL UNIVERSITY

CERTIFICATE

This is to certify that Project-2, Subject Code 203105400 of the 7th Semester, entitled **"CollabriX: A Team-First Freelance Collaboration Platform"** of Group No. PIET CSE _____ has been completed by

- JAGHATHA UDAY PRAKASH – 2303031240447
- BODDU VENKATA SAI KOWSHIK – 2303031241591
- KATREDDI HIMA NAGENDRA – 2303031240520
- IMMADI VENKATA RAMA CHAITANYA VARDHAN – 2303031240436

under my guidance, in partial fulfilment of the Bachelor of Technology (B. Tech) in Computer Science and Engineering at Parul University in the Academic Year 2026–2027.

Date of Submission: _____

Project Guide
Prof. (Dr.) Barkha Makwana

Head of Department,
Dr. Sanjay Agal
Department of Artificial Intelligence and Data Science, PIET
Parul University, Vadodara

Project Coordinator
Dr. Mohammad Arif

External Examiner

Acknowledgements

"Alone we can do so little; together we can do so much."
– Helen Keller

It gives us great pleasure to present this report on **CollabriX**, a platform that is itself about people working better together. Building it taught us as much about collaboration as about software, and we owe that lesson to many people.

First and foremost, we express our sincere gratitude to our project guide, **Prof. (Dr.) Barkha Makwana**, for her constant encouragement, patient reviews and valuable suggestions at every stage of this work. Her questions about scope, security and evaluation shaped the direction of the project and of this report.

We thank **Dr. Sanjay Agal**, Head of the Department of Artificial Intelligence and Data Science, PIET, and **Dr. Mohammad Arif**, Project Coordinator, for providing the facilities, guidelines and timely reviews that made this project possible.

We are grateful to our parents and families for their unwavering support and belief in us, and to our friends and classmates for their constant encouragement.

Finally, we thank all the faculty and staff of Parul Institute of Engineering and Technology who helped us directly or indirectly during the course of this project.

Jaghatha Uday Prakash
Boddu Venkata Sai Kowshik
Katreddi Hima Nagendra
Immadi Venkata Rama Chaitanya Vardhan
B.Tech. – CSE (AI)
Parul University, Vadodara

Abstract

Freelance marketplaces such as Upwork, Fiverr and Freelancer have made it easy for an individual to sell a skill to a client anywhere in the world. They are, however, designed around a single seller competing against other single sellers through bids. Larger projects need a front-end developer, a back-end developer and a designer who already trust each other, and today such teams are assembled informally, outside the platform, through personal messages and spreadsheets. Reputation is also fragile: ratings are often inflated, and they are frequently disconnected from the actual work that was delivered.

CollabriX is a web-based, team-first freelance collaboration platform built to address these gaps. It provides two role-based workspaces. **Developers** can browse open projects, discover other developers by skill, region and rating, send and accept connection requests, chat in real time with their connections, assemble *team batches* with an automatically computed skill-coverage (synergy) score, and climb a ratings-based leaderboard with earned badges. **Clients** can complete a company onboarding, search for talent with faceted filters, message developers, follow their projects on a dashboard and rate completed work.

The system is implemented as a single-page application in **React 18, TypeScript and Vite**, styled with **Tailwind CSS** and **shadcn/ui** components, with **TanStack Query** for server-state caching. The back end is **Supabase**: PostgreSQL with seven application tables, a sign-up trigger that provisions profiles and roles, **Row-Level Security (RLS)** policies that enforce every authorisation rule inside the database, WebSocket-based realtime subscriptions for messages and requests, and object storage for avatars. Ratings can only be inserted for projects that are marked completed and in which the rater actually took part, which ties every review to verifiable work.

The prototype was developed in four one-week agile sprints. It was evaluated through automated browser tests of the authentication flows (9 of 9 test cases passed), direct verification of the RLS policies with an anonymous client (no private table was readable and unauthenticated inserts were rejected with error 42501), static analysis with the TypeScript compiler and ESLint, and a Lighthouse audit of the production build (Performance 78, Accessibility 85, Best Practices 100, SEO 100). The report also documents the current limitations honestly, notably that project posting and status transitions are not yet exposed in the client user interface and that team batches are stored only in the browser, and proposes concrete future enhancements.

Table of Contents

Acknowledgements iii

Abstract iv

Table of Contents v

List of Tables vii

List of Figures viii

List of Algorithms ix

1 Introduction 1

1.1 Drawbacks of the Existing Solution 2

1.2 Scope 2

1.2.1 Key Features and Expected Outcomes 3

1.3 Aim and Objectives 3

1.4 Organisation of the Report 3

2 Literature Survey 4

2.1 Critical Evaluation of Available Literature 4

2.1.1 Paper 1: “Online Labor Markets” by J. J. Horton [1] 4

2.1.2 Paper 2: “The Future of Crowd Work” by A. Kittur, J. V. Nickerson, M. Bernstein et al. [2] 4

2.1.3 Paper 3: “Reputation Systems” by P. Resnick, K. Kuwabara, R. Zeckhauser and E. Friedman [3] 4

2.1.4 Paper 4: “A Survey of Trust and Reputation Systems for Online Service Provision” by A. Jøsang, R. Ismail and C. Boyd [4] 4

2.1.5 Paper 5: “Finding a Team of Experts in Social Networks” by T. Lappas, K. Liu and E. Terzi [5] 5

2.1.6 Paper 6: “Online Team Formation in Social Networks” by A. Anagnostopoulos, L. Becchetti, C. Castillo, A. Gionis and S. Leonardi [11] 5

2.1.7 Paper 7: “Expert Crowdsourcing with Flash Teams” by D. Retelny, S. Robaszkiewicz, A. To et al. [6] 5

2.1.8 Paper 8: “Flash Organizations: Crowdsourcing Complex Work by Structuring Crowds as Organizations” by M. A. Valentine, D. Retelny, A. To et al. [7] 5

2.1.9 Paper 9: “Reputation Inflation” by A. Filippas, J. J. Horton and J. Golden [8] 5

2.1.10 Paper 10: “Reputation Transferability in Online Labor Markets” by M. Kokkodis and P. G. Ipeirotis [9] 6

2.2 Other Platforms Reviewed 6

2.3 Summary of the Literature 6

2.4 Research Gap and Proposed Solution 7

3 Project Flow and Methodology 8

3.1 Software Dependencies 8

3.1.1 Technology Stack 8

3.1.2 Hardware and Software Requirements 8

3.2 Requirement Analysis 9

3.2.1 Intended Users 9
3.2.2 Functional Requirements 10
3.2.3 Non-Functional Requirements 11
3.2.4 User Interface Requirements 11
3.2.5 Assumptions and Constraints 11
3.3 Development Methodology 11
3.3.1 Design Thinking and User-Centric Principles 12
3.3.2 Quality Assurance Strategy 13
3.3.3 Tools for Collaboration 13
3.3.4 Project Timeline (Gantt Chart) 13
3.4 System Architecture 13
3.5 Database Design 14
3.6 UML Design Diagrams 16
3.6.1 Use Case Diagram 16
3.6.2 Sequence Diagram — Registration and Onboarding 17
3.6.3 Sequence Diagram — Connection and Real-time Messaging 18
3.6.4 Navigation Design 18
3.7 User Interface Design 19
3.8 Communication Profile 19
3.9 Security Design 20
3.10 Module Implementation 21
3.10.1 Project Structure 21
3.10.2 Authentication and Role Routing 22
3.10.3 Onboarding and Profiles 22
3.10.4 Developer Discovery, Connections and Requests 22
3.10.5 Real-time Messaging 23
3.10.6 Team Formation Engine 23
3.10.7 Projects, Ratings and Leaderboard 24
3.11 Configuration and Deployment 25
4 Results and Analysis 26
4.1 Tools and Technologies 26
4.2 Results 26
4.2.1 Quantitative Results 28
5 Conclusion and Future Works 33
5.1 Conclusion 33
5.1.1 Lessons Learned 33
5.2 Limitations 34
5.3 Future Prospectus 34
5.3.1 Possible Integration with Other Tools 35
Bibliography 36

List of Tables

1.1 Scope of the CollabriX prototype 2
1.2 Key features of CollabriX and their expected outcomes 3
2.1 Comparison of existing freelance platforms with CollabriX 6
2.2 Summary of reviewed literature 6
3.1 Technology stack and justification 8
3.2 Hardware requirements 8
3.3 Software requirements 9
3.4 Illustrative user personas 9
3.5 Functional requirements 10
3.6 Non-functional requirements 11
3.7 Sprint plan and deliverables (dates from the Git history) 12
3.8 Collaboration and development tools 13
3.9 Architectural layers and responsibilities 14
3.10 Description of database tables 15
3.11 Data dictionary of the profiles table 15
3.12 Communication channels used by CollabriX 19
3.13 Row-Level Security policies implemented 20
3.14 Source code organisation 21
4.1 Tools and technologies used 26
4.2 Comparative analysis of conventional freelance platforms vs. CollabriX 26
4.3 Static analysis results 29
4.4 Automated UI test cases and results 29
4.5 Manual test cases for authenticated features 30
4.6 RLS validation with an anonymous client 31
4.7 Lighthouse metrics and bundle sizes 31
5.1 Project goals versus accomplishments 33
5.2 Known limitations of the current prototype 34

List of Figures

1.1 Overview of the CollabriX collaboration ecosystem 1

3.1 Iterative sprint cycle followed in the project 12

3.2 Gantt chart of the project timeline 13

3.3 Layered system architecture of CollabriX 14

3.4 Entity–relationship diagram 15

3.5 Use case diagram 17

3.6 Sequence diagram for registration and onboarding 17

3.7 Sequence diagram for connection request and messaging 18

3.8 Role-based navigation map 18

3.9 Flowchart of authentication and role-based routing 22

3.10 Flowchart of the synergy score algorithm 24

3.11 Flowchart of rating submission and validation 24

4.1 Splash screen shown while the session is restored 27

4.2 Login screen with e-mail/password and Google sign-in 27

4.3 Sign-up screen with Developer / Client role selection 27

4.4 Developer profile screen with ratings overview and badges 28

4.5 Real-time chat between two connected developers 28

4.6 Team Formation Engine with synergy score and skill gaps 28

4.7 Client dashboard with project statistics 28

4.8 ESLint findings grouped by rule 29

4.9 Sign-up screen rendered at mobile width (TC-09) 30

4.10 Lighthouse category scores for the production build 31

List of Algorithms

1 Synergy Score Calculation for a Team Batch 23
2 Leaderboard Ranking 25

Chapter 1

Introduction

The way software is built has changed. A modern product is rarely written by one person; it is assembled by small cross-functional teams in which a front-end engineer, a back-end engineer, a designer and someone responsible for deployment work together for a few weeks or months. At the same time, an increasing share of this work is performed by independent professionals who are hired online. The freelance or *gig* economy has therefore become an important channel through which companies, especially start-ups and small businesses, access technical talent.

This project, **CollabriX**, is a web platform that connects clients who need software built with developers who build it, and, crucially, lets developers find each other and form teams on the platform itself. The name combines *collaboration* with *X*, indicating that collaboration is the core of the product rather than an afterthought.

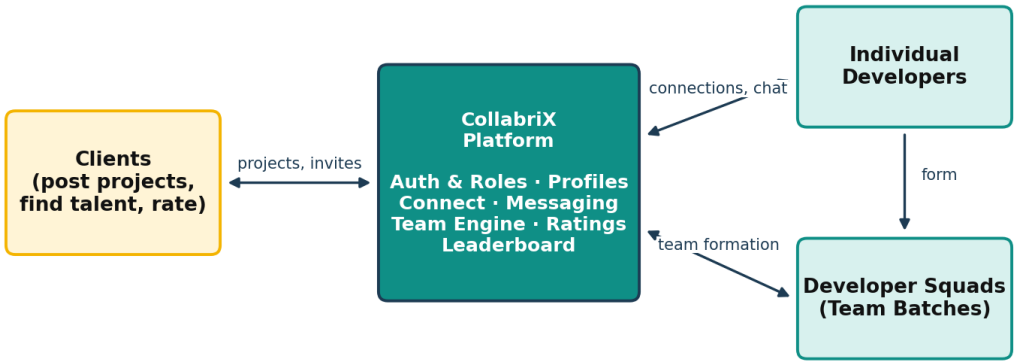


Figure 1.1: Overview of the CollabriX collaboration ecosystem

Online labour markets are two-sided platforms: one side (clients) posts work and the other side (freelancers) offers skills. The platform earns trust by verifying identities, providing communication tools, and maintaining reputation systems such as star ratings and job success scores. Research on online labour markets shows that reputation is the primary signal clients use when choosing whom to hire, and that new entrants without a track record find it hard to win their first contracts [1], [2].

Technically, platforms in this domain are data-intensive, multi-tenant web applications. Every user must only see and modify the data that belongs to them; conversations must be delivered instantly; profile, project and rating data must be consistent. These requirements map naturally onto a relational database with strong access control and a realtime messaging layer, which is the architecture CollabriX adopts.

Student and early-career developers commonly face the same situation: a client asks for a complete application, but no single person can deliver every part alone. Finding a reliable partner usually means asking around in college or social groups, and once a team is formed there is no shared place to talk to the client, track the project or build a joint reputation. Research on *flash teams* and *flash organisations* [6], [7] shows that experts recruited online can deliver complex work when the platform gives them structure. CollabriX applies that idea to everyday freelance development.

The project is relevant to the Indian context in particular, where a large population of young developers is entering the global freelance market and where team-based delivery can help them compete for larger, better-paid contracts.

1.1 Drawbacks of the Existing Solution

1. **Individual-only model:** mainstream marketplaces are optimised for one freelancer bidding on one job. There is no first-class way for developers to discover complementary peers and apply to a project as a team.
2. **Bidding fatigue:** clients receive many generic proposals and must filter them manually, while freelancers spend unpaid time writing bids.
3. **Weak link between ratings and work:** reviews tend to be inflated over time [8] and are not always tied in a verifiable way to a completed engagement, which reduces their value as a quality signal.
4. **Fragmented communication:** conversations, team coordination and project status often move to external tools such as e-mail or chat apps, so the platform loses context.
5. **Cold-start problem:** newcomers without reviews are almost invisible in search results, regardless of their actual skills [2].

1.2 Scope

The scope of the project is a working web prototype of the platform, including both user roles, the developer networking features and the supporting database and security layer. Table 1.1 summarises what is included and what is deliberately excluded.

Table 1.1: Scope of the CollabriX prototype

In scope	Out of scope (for this phase)
Role-based sign-up and login (e-mail/password and Google OAuth)	Payments, escrow and invoicing
Developer and client onboarding wizards and profile editing	Native Android / iOS applications
Developer search with type, skill and region filters	Automated résumé or LinkedIn parsing
Connection requests and real-time one-to-one messaging	Video calls and file sharing in chat
Team batch formation with synergy scoring	Activity tracking / screenshot monitoring
Completion-gated project and user ratings, leaderboard, badges	Dispute resolution and moderation tools
Row-Level Security for every application table	Multi-language (i18n) support

1.2.1 Key Features and Expected Outcomes

Table 1.2: Key features of CollabriX and their expected outcomes

Feature	Description	Expected outcome
Role-based workspaces	Separate bottom-navigation and screens for developers (5 tabs) and clients (4 tabs)	Each user sees only relevant tools
Guided onboarding	Three-step developer and company setup wizards with live preview	Complete profiles from day one
Developer discovery	Search by name, skill, type; continent → country region filter; rating badge	Faster peer and talent discovery
Connections & messaging	Request/accept workflow, unread badges, realtime chat threads	In-platform communication
Team Formation Engine	Batches built from accepted connections with role assignment and synergy %	Balanced, complementary teams
Ratings & leaderboard	1–5 star ratings tied to completed projects; ranking and badges	Trustworthy reputation signal
Security by design	RLS policies, JWT sessions, per-user storage folders	Zero-trust data access

1.3 Aim and Objectives

1. To design and implement a responsive single-page web application with separate, role-specific workspaces for developers and clients.
2. To enable developers to discover peers by skill, developer type and region, and to connect with them through a request–accept workflow.
3. To provide real-time messaging between connected users without polling.
4. To let developers assemble team batches from their verified connections and see an objective skill-coverage score for the team.
5. To build a reputation system in which ratings can only be submitted for completed projects by the people who took part in them.
6. To enforce all authorisation rules at the database level using PostgreSQL Row-Level Security, so that the browser never has to be trusted.
7. To evaluate the system through functional, security and performance testing and to document its limitations.

1.4 Organisation of the Report

Chapter 2 evaluates related research and existing platforms. Chapter 3 covers the project flow and methodology, from requirements and design to the implementation of each module. Chapter 4 presents the tools used, screenshots and test results. Chapter 5 concludes the report and outlines future work.

Chapter 2

Literature Survey

This chapter reviews research on online labour markets, reputation systems and team formation, followed by an analysis of popular freelance platforms. The goal is to identify what is already known, what existing systems do well, and where a team-first platform can contribute.

2.1 Critical Evaluation of Available Literature

2.1.1 Paper 1: "Online Labor Markets" by J. J. Horton [1]

Horton describes online labour markets as markets in which workers and buyers are matched over the internet and work is delivered electronically. He discusses their economic properties: low search costs, global participation, and the strong role that the platform plays as a market designer. A key observation is that the platform's rules — how jobs are posted, how workers are ranked and how feedback is collected — directly shape outcomes for both sides.

Relevance: CollabriX treats these design choices (who may message whom, who may rate whom) as explicit, enforced rules rather than conventions.

2.1.2 Paper 2: "The Future of Crowd Work" by A. Kittur, J. V. Nickerson, M. Bernstein et al. [2]

Kittur and colleagues analyse paid crowd work and propose a research agenda for making it a career that people would want for their children. They identify the need for better task decomposition, worker collaboration, reputation and credentialing, and support for complex, interdependent work rather than isolated micro-tasks.

Relevance: the paper motivates CollabriX's focus on collaboration between workers and on reputation that reflects real outcomes.

2.1.3 Paper 3: "Reputation Systems" by P. Resnick, K. Kuwabara, R. Zeckhauser and E. Friedman [3]

This influential article explains how reputation systems build trust between strangers by collecting, distributing and aggregating feedback about past behaviour. The authors note three challenges: people may not leave feedback, negative feedback is hard to elicit, and it is hard to ensure that feedback is honest. They argue that the system must make feedback both available and credible.

Relevance: CollabriX addresses credibility by making it impossible, at the database level, to rate a user or project unless a completed engagement exists.

2.1.4 Paper 4: "A Survey of Trust and Reputation Systems for Online Service Provision" by A. Jøsang, R. Ismail and C. Boyd [4]

Jøsang, Ismail and Boyd survey trust and reputation models used in online services, from simple summation and averaging (as on eBay) to Bayesian and flow-based models. They also

catalogue attacks such as ballot stuffing, bad-mouthing and identity changes, and discuss how system design can mitigate them.

Relevance: averaging is used in the prototype for simplicity; unique constraints prevent the same rater from rating the same project twice, which mitigates ballot stuffing.

2.1.5 Paper 5: "Finding a Team of Experts in Social Networks" by T. Lappas, K. Liu and E. Terzi [5]

Lappas, Liu and Terzi formalise the *team formation problem*: given a task requiring a set of skills and a social network of experts, find a team that covers all required skills while minimising the communication cost between members. They prove the problem is NP-hard and propose approximation algorithms based on the diameter and minimum spanning tree of the team in the network.

Relevance: CollabriX restricts team members to a developer's accepted connections (existing social ties) and measures skill coverage against domain requirements — a practical, explainable version of this formulation.

2.1.6 Paper 6: "Online Team Formation in Social Networks" by A. Anagnostopoulos, L. Becchetti, C. Castillo, A. Gionis and S. Leonardi [11]

This work extends team formation to an online setting in which tasks arrive over time and the system must assign teams while balancing the workload of experts. The authors show that good solutions must consider both skill coverage and fairness of load across people.

Relevance: the profiles table already stores a `current_projects` count, which can be used in future to balance workload during team suggestions.

2.1.7 Paper 7: "Expert Crowdsourcing with Flash Teams" by D. Retelny, S. Robaszkiewicz, A. To et al. [6]

Flash teams are computationally guided teams of crowd experts assembled on demand. Using a framework called Foundry, the authors modelled workflows as linked modular tasks and recruited experts from an online marketplace; flash teams completed complex tasks such as app design considerably faster than traditional self-managed teams.

Relevance: demonstrates that structured, on-platform team assembly can outperform ad hoc coordination, which is the core hypothesis of CollabriX.

2.1.8 Paper 8: "Flash Organizations: Crowdsourcing Complex Work by Structuring Crowds as Organizations" by M. A. Valentine, D. Retelny, A. To et al. [7]

Flash organisations extend flash teams with organisational structures — roles, hierarchies and the ability to change the structure while work is in progress. Crowd workers in these organisations built complex products such as a mobile application. The work highlights the importance of explicit roles.

Relevance: CollabriX lets the batch creator assign explicit roles (Team Lead, Frontend Engineer, Backend Engineer, etc.) to each member.

2.1.9 Paper 9: "Reputation Inflation" by A. Filippas, J. J. Horton and J. Golden [8]

Using data from a large online labour market, the authors show that average feedback scores rose substantially over time while underlying quality did not, because giving negative

feedback is costly to the rater. As a result, ratings lose their ability to distinguish good from average workers.

Relevance: motivates multi-dimensional and completion-tied ratings, and future work on private (double-blind) feedback in CollabriX.

2.1.10 Paper 10: "Reputation Transferability in Online Labor Markets" by M. Kokkodis and P. G. Ipeirotis [9]

Kokkodis and Ipeirotis study how a worker's reputation in one category of work predicts performance in another, and propose models that transfer reputation across task categories. They find that naive aggregation can mislead employers when workers switch domains.

Relevance: supports showing skill-specific information (skills, developer type) alongside a single average rating.

2.2 Other Platforms Reviewed

Four widely used platforms were studied from the perspective of both a client and a developer. Table 2.1 summarises the comparison against the features that matter for team-based delivery.

Table 2.1: Comparison of existing freelance platforms with CollabriX

Aspect	Upwork	Fiverr	Freelancer	Toptal	CollabriX
Hiring model	Job post + proposals	Fixed gigs / packages	Bidding + contests	Curated matching	Profiles + teams
Peer discovery between freelancers	Limited	No	Limited	No	Yes (search + connect)
Form a team on-platform	Agencies only	No	No	Matched by staff	Yes (team batches)
Skill-coverage analysis	No	No	No	Manual	Synergy score
In-platform chat	Yes	Yes	Yes	Yes	Yes (realtime)
Ratings tied to completed work	Yes	Yes	Yes	N/A	Enforced by RLS
Open leaderboard	No	Seller levels	Rankings	No	Yes + badges

2.3 Summary of the Literature

Table 2.2: Summary of reviewed literature

Ref.	Focus	Key finding	Gap addressed by CollabriX
[1]	Online labour markets	Platform rules shape market outcomes	Rules enforced in the database
[2]	Crowd work agenda	Need collaboration & reputation	Team-first design
[3], [4]	Reputation systems	Feedback must be credible; attacks exist	Completion-gated, unique ratings

Ref.	Focus	Key finding	Gap addressed by CollabriX
[5], [11]	Team formation	Cover skills, low communication cost	Teams from existing connections + coverage score
[6], [7]	Flash teams / organisations	Structured expert teams deliver complex work	Explicit roles in batches
[8]	Reputation inflation	Average scores drift upward	Basis for future double-blind ratings
[9]	Reputation transfer	Category matters	Skills shown with ratings

2.4 Research Gap and Proposed Solution

The literature establishes that (i) structured teams of online experts can deliver complex work, (ii) team formation should consider both skills and social ties, and (iii) reputation must be credible to be useful. Existing commercial platforms, however, remain largely individual-centric and do not let freelancers discover peers, form teams and evaluate their skill coverage within the platform. CollabriX fills this gap with a connection-based team formation engine, completion-tied ratings enforced in the database, and integrated real-time communication.

Chapter 3

Project Flow and Methodology

This project aims to design and implement a team-first freelance collaboration platform. To accomplish this goal, we followed a systematic approach consisting of requirement analysis, design, iterative development, testing and evaluation. This chapter describes the software dependencies, the requirements, the development methodology, the architecture and database design, the communication and security model, and the implementation of each module.

3.1 Software Dependencies

CollabriX is a pure software project: it needs no special hardware beyond a development computer, and end users only need a modern web browser. The software stack was chosen to minimise the amount of custom server code while keeping strong typing and security.

3.1.1 Technology Stack

Table 3.1: Technology stack and justification

Layer	Technology	Reason for selection
Language	TypeScript 5.8	Static typing catches errors early; generated database types give end-to-end type safety
UI library	React 18.3	Component model, huge ecosystem, team familiarity
Build tool	Vite 5.4 + SWC	Instant dev server start and fast production builds (≈ 2 s)
Styling	Tailwind CSS 3.4 + shadcn/ui	Design tokens, dark mode, accessible Radix primitives
Server state	TanStack Query 5	Caching, background refetch, mutation invalidation
Routing	React Router 6	Declarative routes and redirects
Validation	Zod 3	Schema-based validation of forms
Back-end	Supabase (PostgreSQL)	Auth, REST, realtime and storage without writing a server; RLS for security
Icons / charts	Lucide, Recharts	Consistent icon set; charting for dashboards

Alternatives considered were the MERN stack (MongoDB, Express, React, Node) and Firebase. MERN would have required writing and hosting a custom API and authorisation layer; Firebase uses a document database whose security rules are less expressive for relational data such as connections and ratings. Supabase offered a relational model, SQL-based security policies and realtime in one managed service.

3.1.2 Hardware and Software Requirements

Table 3.2: Hardware requirements

Component	Development	End user
Processor	Intel Core i5 / Apple M1 or better	Any modern dual-core CPU or smartphone
Memory	8 GB RAM (16 GB recommended)	2 GB RAM
Storage	2 GB free (Node modules, tooling)	No installation required
Network	Broadband internet	Internet connection (3G or better)

Table 3.3: Software requirements

Category	Software / version
Operating system	macOS, Windows 10+ or Linux (development); any OS with a modern browser (users)
Runtime & package manager	Node.js 18+ and npm 9+
Front-end	React 18.3, TypeScript 5.8, Vite 5.4, Tailwind CSS 3.4, shadcn/ui (Radix UI), TanStack Query 5, React Router 6, Zod 3
Back-end	Supabase (PostgreSQL, Auth, Realtime, Storage), supabase-js 2.88
Browser	Chrome, Edge, Firefox or Safari (latest two versions)
Tools	VS Code, Git and GitHub, Lovable, Supabase Studio, ESLint, Chrome DevTools, Lighthouse

3.2 Requirement Analysis

The purpose of this specification is to describe the functional and non-functional requirements of the CollabriX web application in sufficient detail for design, implementation and testing. It is intended for the project team, the project guide and the evaluators.

3.2.1 Intended Users

CollabriX has two primary user classes and one supporting actor:

- **Developer** — an individual freelance developer, designer or engineer who wants to find projects, connect with peers, form teams and build a reputation.
- **Client** — a company representative, start-up founder or individual who needs software built and wants to find and communicate with talent.
- **Supabase platform** — the managed backend that authenticates users, stores data, enforces security policies and pushes realtime events.

The illustrative personas in the following table were used to keep design decisions focused on real needs.

Table 3.4: Illustrative user personas

Persona	Goals	Pain points today
Riya, final-year student developer	Win first paid projects; work with peers	No reviews yet; can't find a back-end partner

Persona	Goals	Pain points today
Arjun, freelance full-stack engineer	Take larger contracts with a reliable team	Coordinates teams over personal chat apps
Meera, start-up founder (client)	Hire a small team for an MVP quickly	Too many generic proposals; hard to judge
Rahul, SME IT manager (client)	Find specialists by region and skill	Little trust in inflated ratings

The product scope is a responsive web application usable on desktop and mobile browsers. It covers account management, profiles, discovery, connections, messaging, team formation, ratings and leaderboard, as listed in Table 1.1.

3.2.2 Functional Requirements

Table 3.5: Functional requirements

ID	Requirement	Priority
FR-1	The system shall allow a user to register with full name, e-mail, password and a role (developer or client).	High
FR-2	The system shall allow login with e-mail/password and with Google OAuth.	High
FR-3	The system shall automatically create a profile and role record when a user registers.	High
FR-4	The system shall guide new developers and clients through a three-step onboarding wizard.	Medium
FR-5	The system shall route a logged-in user to the developer or client workspace based on the stored role.	High
FR-6	Developers shall be able to edit profile details, avatar, skills, region and a cover image.	Medium
FR-7	Users shall be able to search developers by name, skill and type, and filter by continent and country.	High
FR-8	A developer shall be able to send a connection request; the receiver shall accept or reject it.	High
FR-9	Connected users shall exchange messages delivered in real time.	High
FR-10	The system shall show unread request and message counts as badges.	Medium
FR-11	Developers shall create team batches from accepted connections, assign roles and view a synergy score.	High
FR-12	Developers shall browse open projects and view projects assigned to them.	Medium
FR-13	Clients shall view a dashboard of their projects, budget totals and hired developers.	Medium
FR-14	Participants of a completed project shall rate the project and the other party (1–5 stars, optional review).	High
FR-15	The system shall rank developers on a leaderboard and award badges based on ratings and activity.	Low
FR-16	Users shall be able to sign out from any screen.	High

3.2.3 Non-Functional Requirements

Table 3.6: Non-functional requirements

ID	Category	Requirement
NFR-1	Security	Every application table shall have Row-Level Security enabled; unauthenticated clients shall not read private data.
NFR-2	Security	Sessions shall use signed JWTs; secrets other than the public anon/ publishable key shall never reach the browser.
NFR-3	Performance	The login page should reach Lighthouse Performance ≥ 75 and Best Practices ≥ 90 on the production build.
NFR-4	Real-time	Messages and requests shall be pushed over WebSockets; no periodic polling.
NFR-5	Usability	The interface shall be mobile-first, usable at 375 px width without horizontal scrolling.
NFR-6	Accessibility	Text and controls should meet WCAG 2.1 AA contrast where possible; forms have labels.
NFR-7	Maintainability	Code shall be written in TypeScript and pass the TypeScript compiler without errors.
NFR-8	Portability	The front end shall be a static build deployable to any CDN; configuration via environment variables.
NFR-9	Reliability	Network failures shall be reported to the user with clear messages instead of silent failures.

3.2.4 User Interface Requirements

- A consistent teal colour theme with light and dark modes, Inter font and rounded card components.
- A fixed bottom navigation bar with icons and labels; the active tab is highlighted.
- A sticky header with logo, theme toggle, notification bell with unread count, and account menu.
- Feedback for every action through toast notifications (success and error).
- Empty states with an icon and explanatory text whenever a list has no data.

3.2.5 Assumptions and Constraints

- Users have a valid e-mail address; Google login requires the Google provider to be enabled in Supabase.
- The free tier of Supabase limits database size and realtime connections, which is sufficient for a prototype.
- Payments are out of scope; budgets are informational only.

3.3 Development Methodology

The project followed an **Agile, iterative approach** inspired by Scrum [24]. After an initial phase of requirement study, literature survey and design, development proceeded in four one-week sprints. Each sprint started with selecting items from a prioritised backlog, continued

with design and implementation, and ended with a demo, testing and a commit to the shared Git repository. Agile was preferred over the Waterfall model because requirements — especially around onboarding and team formation — evolved as each sprint build was reviewed.

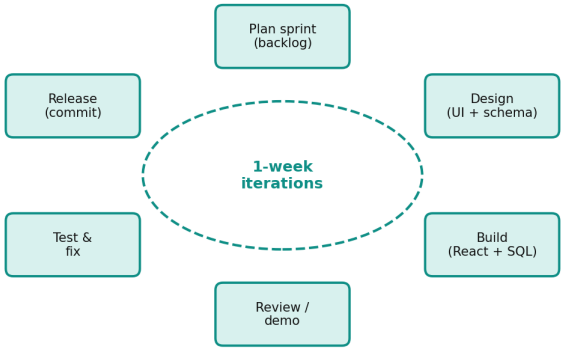


Figure 3.1: Iterative sprint cycle followed in the project

Table 3.7: Sprint plan and deliverables (dates from the Git history)

Sprint	Period	Main deliverables
Sprint 1	16 – 23 Aug 2026	Supabase configuration and fallbacks, .env setup, authentication fixes, role selection
Sprint 2	24 – 31 Aug 2026	Connected messaging and ratings, role-specific profile titles and talent filters, role in sign-up metadata
Sprint 3	1 – 5 Sep 2026	Login page redesign, Google sign-in fix, developer and client onboarding, removal of mock data, cover images
Sprint 4	4 – 17 Sep 2026	Team batch formation engine, CollabriX vs Upwork comparison, configuration restore
Stabilisation	17 – 22 Sep 2026	Bug fixes and build verification

3.3.1 Design Thinking and User-Centric Principles

We applied the five stages of design thinking:

1. **Empathise** — studying how freelancers and clients use existing platforms (Chapter 2) and listing the frustrations of both sides, such as bidding fatigue and finding trustworthy partners.
2. **Define** — the core problem statement: *"Freelance developers cannot easily find trusted partners and deliver as a team on the same platform where clients hire them."*
3. **Ideate** — brainstorming produced ideas such as squads, synergy scores, completion-gated ratings and a public leaderboard.
4. **Prototype** — the first clickable prototype was generated with Lovable and then refined by hand in code.
5. **Test** — each sprint build was exercised by the team in both roles; the findings led to changes such as separating client and developer profile titles (commit of 30 Aug 2026) and removing mock data (2 Sep 2026).

3.3.2 Quality Assurance Strategy

- **Static analysis:** the TypeScript compiler (`tsc --noEmit`) and ESLint with React Hooks rules on every significant change.
- **Automated UI tests:** headless Chrome driven by Puppeteer to exercise authentication flows and responsive layout.
- **Security verification:** direct queries with an anonymous Supabase client to confirm that RLS denies private reads and unauthenticated writes.
- **Performance audit:** Google Lighthouse on the production build.
- **Manual acceptance testing:** two-account scenarios for connections, messaging, team batches and ratings (Section 4.2.1).

3.3.3 Tools for Collaboration

Table 3.8: Collaboration and development tools

Tool	Usage
Git and GitHub	Version control; 102 commits; history used to reconstruct sprint dates
Lovable	AI-assisted generation of the initial UI scaffold and quick iterations
Visual Studio Code	Primary code editor with ESLint and Tailwind extensions
Supabase Studio	Database tables, SQL migrations, auth provider and storage configuration
Puppeteer + headless Chrome	Automated UI test cases and screenshots (Chapter 4)
Chrome DevTools / Lighthouse	Debugging, network inspection and performance audits

3.3.4 Project Timeline (Gantt Chart)

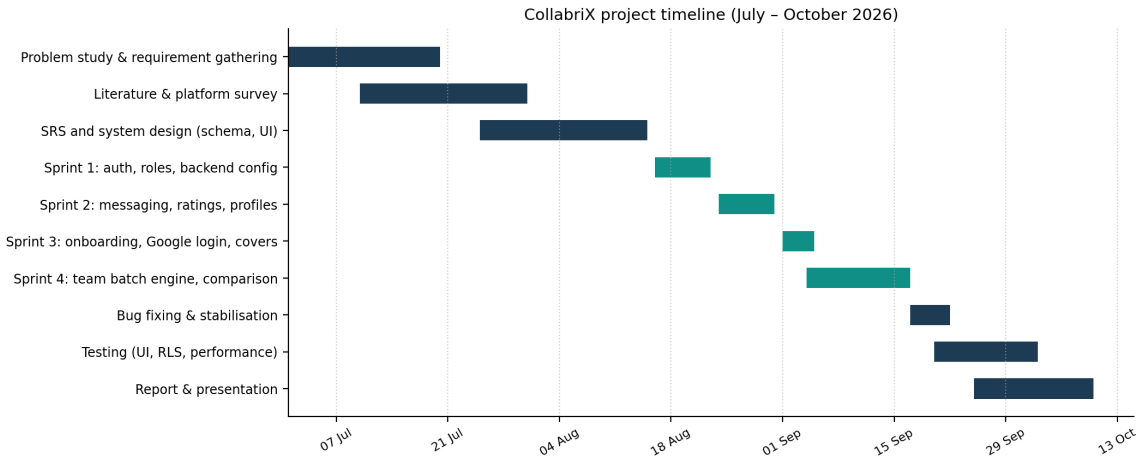


Figure 3.2: Gantt chart of the project timeline

3.4 System Architecture

CollabriX follows a **client–server architecture with a backend-as-a-service (BaaS)**. The client is a single-page application that runs entirely in the browser. Instead of a custom application server, it talks directly to Supabase, which exposes the PostgreSQL database

through an auto-generated REST API (PostgREST), authenticates users with JWTs, and broadcasts database changes over WebSockets. Business rules about *who may do what* are implemented as Row-Level Security policies and triggers inside PostgreSQL, so they apply no matter which client sends the request.

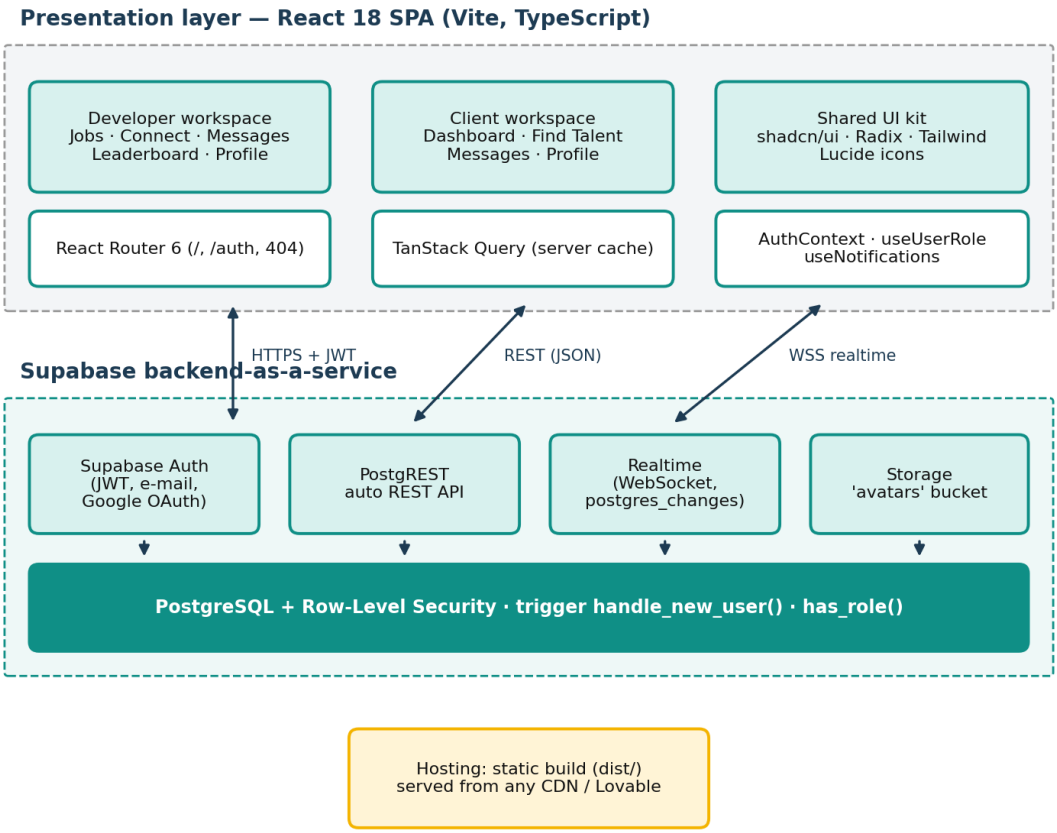


Figure 3.3: Layered system architecture of CollabriX

Table 3.9: Architectural layers and responsibilities

Layer	Components	Responsibility
Presentation	Pages (Index, Auth, NotFound), 10 screens, 20 UI primitives	Rendering, navigation, form validation
State	AuthContext, useUserRole, useNotifications, TanStack Query	Session, role, unread counts, cached server data
API	supabase-js → PostgREST, Auth, Realtime, Storage	HTTP and WebSocket communication
Data & security	7 tables, RLS policies, trigger, SQL functions	Persistence, integrity, authorisation

3.5 Database Design

The database consists of seven application tables in the `public` schema plus the `auth.users` table managed by Supabase. Every profile row shares its primary key with the corresponding `auth.users` row, which makes `auth.uid()` directly comparable with profile identifiers inside RLS policies. The entity–relationship diagram is shown below.

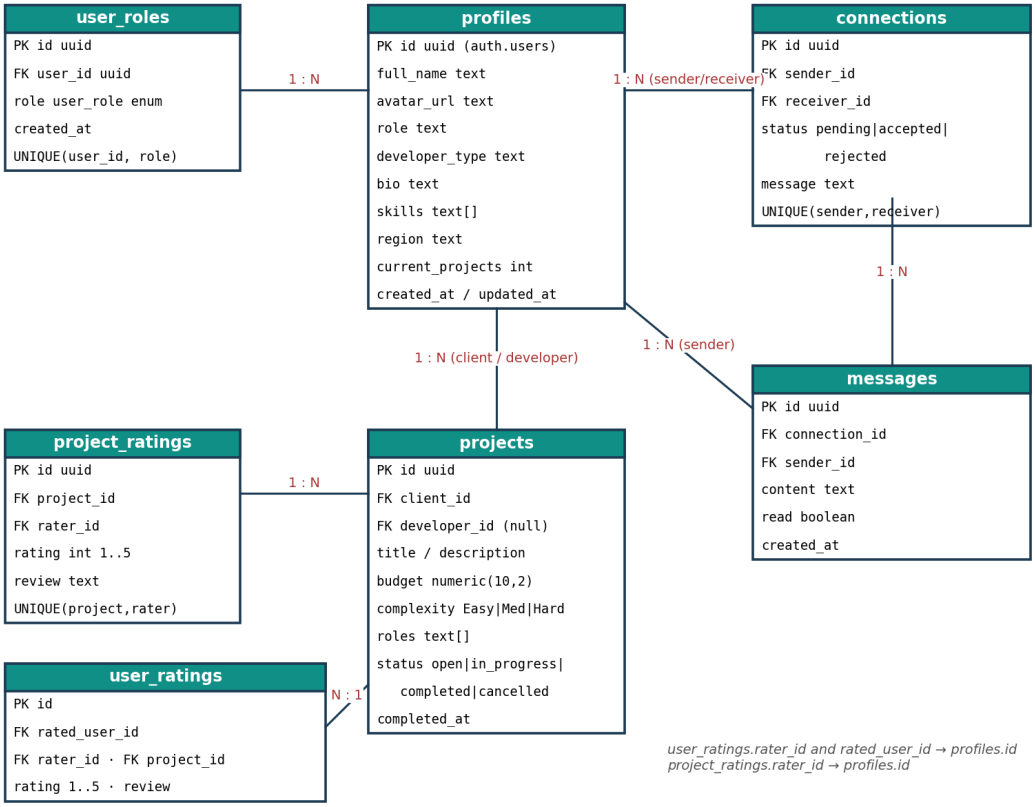


Figure 3.4: Entity–relationship diagram

Table 3.10: Description of database tables

Table	Purpose	Key constraints
profiles	Public identity: name, avatar, bio, role title, developer type, skills, region	PK = auth.users.id (ON DELETE CASCADE)
user_roles	Authorisation role: developer or client (enum user_role)	UNIQUE(user_id, role); one role per user via RLS
connections	Connection requests between two profiles	status ∈ {pending, accepted, rejected}; UNIQUE(sender, receiver)
messages	Chat messages inside a connection	FK connection_id; read flag; realtime enabled
projects	Work posted by a client, optionally assigned to a developer	complexity ∈ {Easy, Medium, Hard}; status ∈ {open, in_progress, completed, cancelled}
project_ratings	Rating of a project by a participant	rating 1–5; UNIQUE(project_id, rater_id)
user_ratings	Rating of one participant by another	rating 1–5; UNIQUE(rated_user, rater, project)

The following table gives the data dictionary of the central `profiles` table, which is read by almost every screen.

Table 3.11: Data dictionary of the profiles table

Column	Type	Default / constraint	Description
id	uuid	PK, FK → auth.users(id) ON DELETE CASCADE	Same identifier as the authentication account
full_name	text	from sign-up metadata, else 'New User'	Person's name, or company name for clients
avatar_url	text	nullable	Public URL of the image in the avatars bucket
role	text	'Full Stack Developer' or 'Client'	Headline shown on cards; used to hide clients from talent search
developer_type	text	'Full Stack Developer'; NULL for clients	Specialisation used by type filters
bio	text	nullable	Free-text professional summary
skills	text[]	'{}'	Skill tags used for search and synergy scoring
region	text	nullable	Continent or country used by the region filter
current_projects	integer	NOT NULL DEFAULT 0	Workload counter reserved for team suggestions
created_at / updated_at	timestamptz	now(); trigger updates updated_at	Audit timestamps

3.6 UML Design Diagrams

3.6.1 Use Case Diagram

The use case diagram below shows the main use cases. Developers and clients share messaging, profile editing and rating; developer-only use cases cover networking and team formation, while client-only use cases cover company onboarding, talent search and the dashboard.

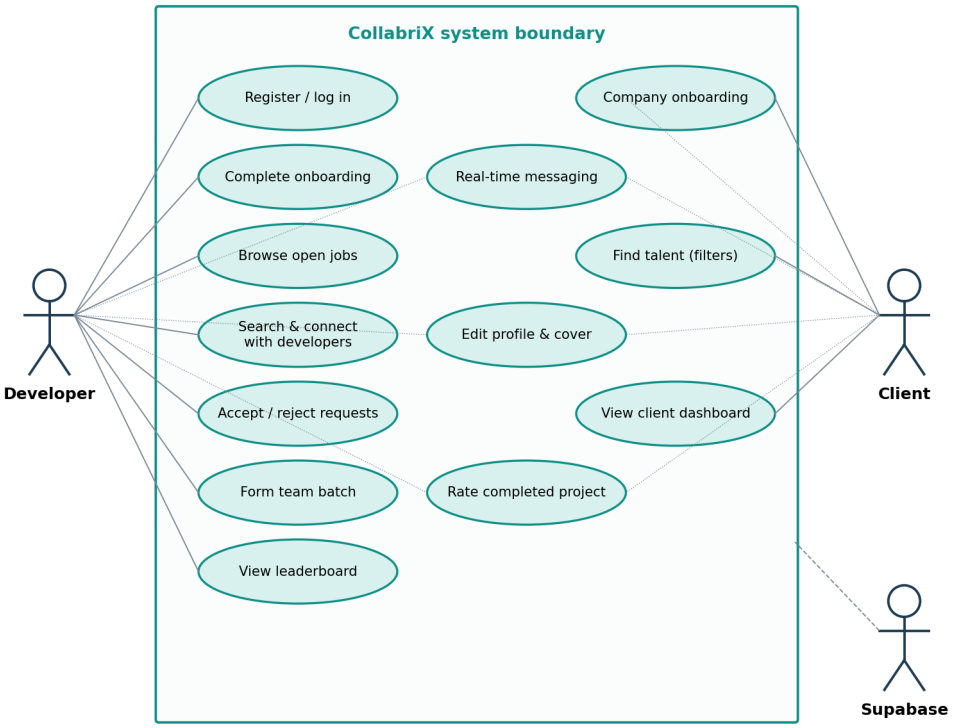


Figure 3.5: Use case diagram

3.6.2 Sequence Diagram — Registration and Onboarding

The sequence diagram below shows the sign-up flow. The role is passed as user metadata so that the database trigger `handle_new_user()` can create both the profile and the role record atomically, before the client even receives the session.

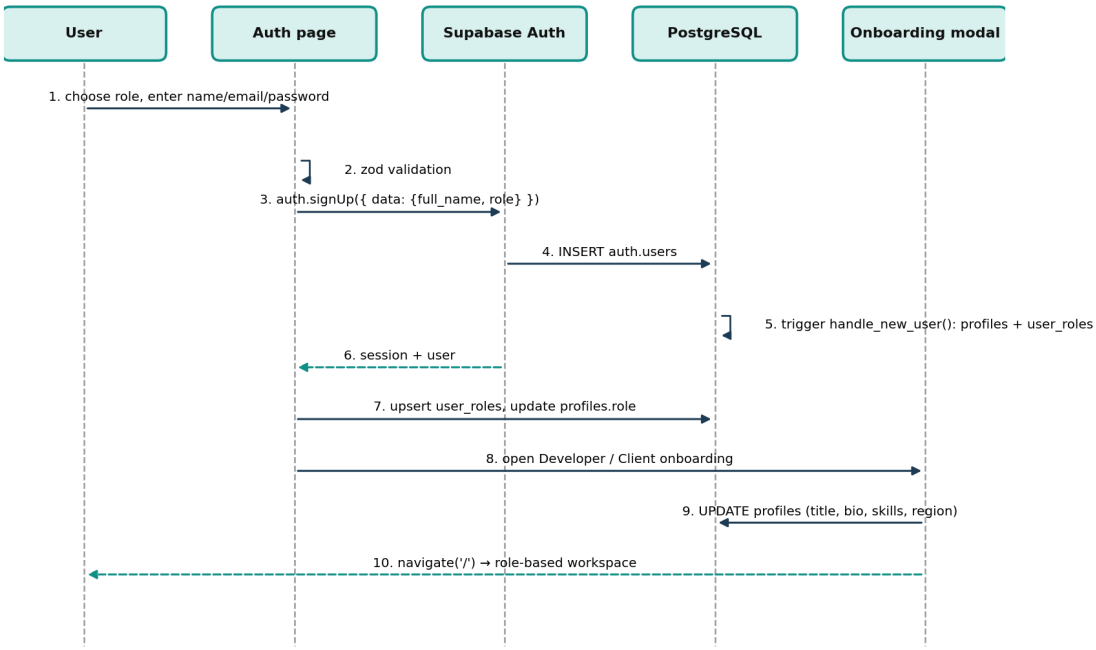


Figure 3.6: Sequence diagram for registration and onboarding

3.6.3 Sequence Diagram — Connection and Real-time Messaging

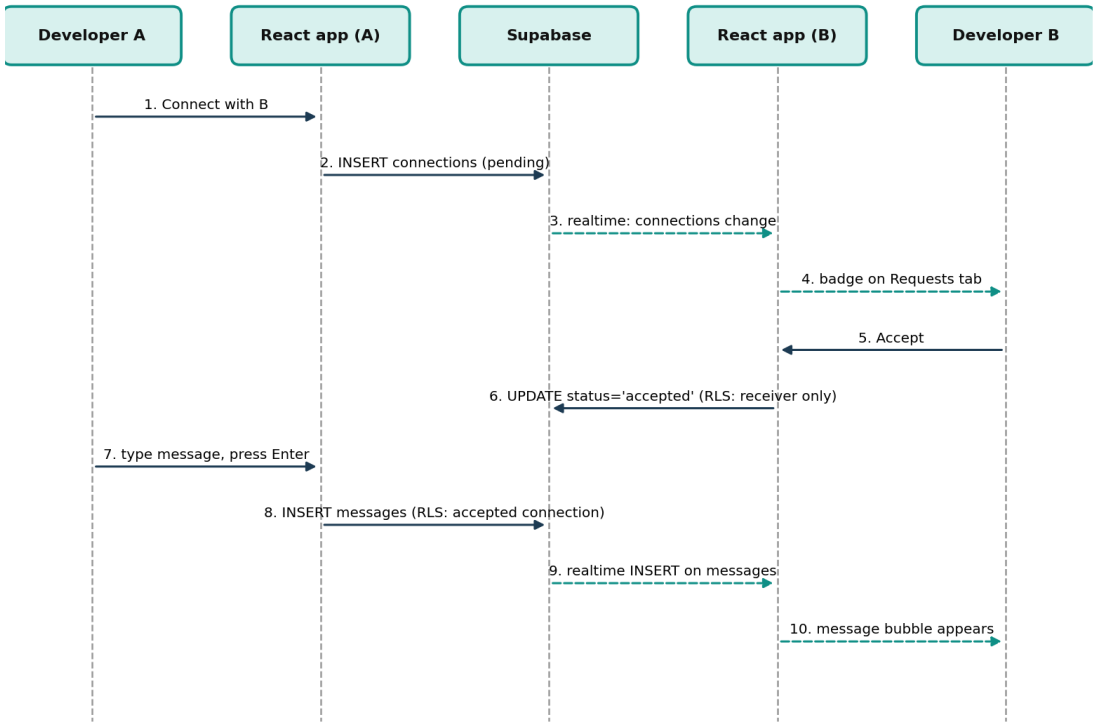


Figure 3.7: Sequence diagram for connection request and messaging

3.6.4 Navigation Design

Navigation is driven by a single route (/) whose content depends on the active tab and the user's role. This keeps transitions instant (no page reloads) and makes the application feel like a native mobile app.

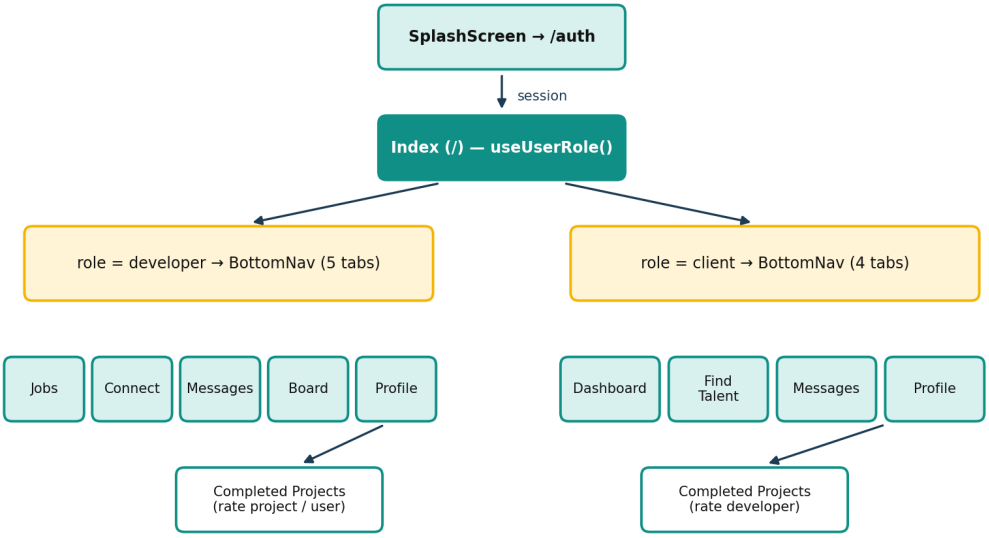


Figure 3.8: Role-based navigation map

3.7 User Interface Design

The interface follows a **mobile-first, component-driven** approach. Design tokens (colours, radii, shadows, gradients) are defined once as CSS custom properties in `index.css` and consumed through Tailwind utility classes, so the whole application can switch between light and dark themes by toggling a single class. The primary colour is teal (HSL 175° 80% 35%), chosen to convey trust and calm. Components such as buttons, dialogs, tabs and selects come from shadcn/ui, which wraps accessible Radix UI primitives — keyboard navigation and ARIA attributes are therefore built in.

- **Consistency:** every screen uses the same card, badge and empty-state patterns.
- **Feedback:** loading spinners for every fetch, and toast messages for every mutation.
- **Progressive disclosure:** filters are hidden behind a filter button; onboarding is split into three short steps.

3.8 Communication Profile

CollabriX uses two communication channels between the browser and the backend. Request–response operations (reading profiles, inserting a message, updating a connection) travel over HTTPS to PostgREST with the user's JWT in the Authorization header [14]. Change notifications travel the other way over a single multiplexed WebSocket connection to Supabase Realtime [13], on which the client opens one channel per subscription. The following table summarises the channels used.

Table 3.12: Communication channels used by CollabriX

Channel	Protocol	Direction	Used for
PostgREST API	HTTPS + JSON, JWT	Client → server	All reads and writes on application tables
Supabase Auth	HTTPS + JSON	Client ↔ server	Sign-up, login, OAuth redirect, token refresh
Realtime: messages-<id>	WebSocket	Server → client	New messages in an open chat thread
Realtime: connection-requests	WebSocket	Server → client	New or updated requests for the receiver
Realtime: notifications-*	WebSocket	Server → client	Unread badge counts in header and navigation
Storage API	HTTPS (multipart)	Client → server	Avatar upload; public URL for display

- **Caching:** TanStack Query caches project and rating queries and invalidates them after mutations, avoiding repeated network calls.
- **Realtime instead of polling:** Supabase channels subscribe to `postgres_changes` filtered by connection or receiver, so only relevant events reach each client.
- **Embedded joins:** a single PostgREST request fetches a project together with its client, developer and ratings using foreign-key embedding.

- **Integration:** Google OAuth via Supabase Auth; Supabase Storage for avatars; environment variables (`VITE_SUPABASE_URL`, `VITE_SUPABASE_PUBLISHABLE_KEY`) for configuration.

3.9 Security Design

Security is designed on a zero-trust principle: the browser holds only a public (publishable) key, and the database decides on every row. The following table lists the policies implemented in the migrations.

Table 3.13: Row-Level Security policies implemented

Table	Operation	Policy (who is allowed)
profiles	SELECT / INSERT / UPDATE	Everyone may read; a user may insert and update only the row where id = auth.uid()
user_roles	SELECT / INSERT	Read own role; insert only developer/client and only if the user has no role yet
connections	SELECT / INSERT / UPDATE / DELETE	Parties may read; sender inserts; only the receiver may update status; sender may delete
messages	SELECT / INSERT / UPDATE	Only members of an accepted connection may read or send; sender may update own messages
projects	SELECT / INSERT / UPDATE / DELETE	Anyone may read; only the owning client may create, update or delete
project_ratings	SELECT / INSERT	Anyone may read; insert only by a participant of a completed project
user_ratings	SELECT / INSERT	As above, and a user cannot rate themselves
storage: avatars	INSERT / UPDATE / DELETE	Only inside the folder named after the user's own id

Additional measures include input validation with Zod on the client, parameterised queries through PostgREST (eliminating SQL injection), React's automatic output escaping (mitigating XSS), and `SECURITY DEFINER` functions with a fixed `search_path` to prevent search-path hijacking.

The back end is defined entirely in SQL migrations, which makes it reproducible on any Supabase project. The most important piece is the trigger that runs when a new user is inserted into `auth.users`. It reads the role from the sign-up metadata and provisions both the profile and the role record:

Listing 3.1: Sign-up trigger (abridged)

```
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE user_role_val public.user_role; raw_role text;
BEGIN
  raw_role := NEW.raw_user_meta_data ->> 'role';
  IF raw_role = 'client' THEN user_role_val := 'client';
  ELSE user_role_val := 'developer'; END IF;
  INSERT INTO public.profiles (id, full_name, avatar_url, role, developer_type)
  VALUES (NEW.id, COALESCE(NEW.raw_user_meta_data ->> 'full_name', 'New User'), ...)
  ON CONFLICT (id) DO UPDATE SET full_name = EXCLUDED.full_name, ...;
  INSERT INTO public.user_roles (user_id, role) VALUES (NEW.id, user_role_val)
  ON CONFLICT (user_id, role) DO NOTHING;
  RETURN NEW;
END; $$;
```

Ratings are protected by a policy that checks project participation and completion status in the same statement:

Listing 3.2: Completion-gated rating policy

```
CREATE POLICY "Users can rate others from completed projects"
ON public.user_ratings FOR INSERT WITH CHECK (
  auth.uid() = rater_id AND auth.uid() != rated_user_id AND
  EXISTS (SELECT 1 FROM public.projects p
          WHERE p.id = project_id
            AND (p.client_id = auth.uid() OR p.developer_id = auth.uid())
            AND p.status = 'completed'));
```

The `messages` and `connections` tables are added to the `supabase_realtime` publication so that inserts and updates are broadcast to subscribed clients, and an `updated_at` trigger keeps modification timestamps accurate on profiles, connections and projects.

3.10 Module Implementation

3.10.1 Project Structure

The source code contains approximately 9,700 lines of TypeScript/TSX in the `src` folder and eight SQL migration files. The following table summarises the main folders.

Table 3.14: Source code organisation

Folder	Contents
src/pages	Index (role-based shell), Auth (login / sign-up), NotFound
src/screens	JobsScreen, ConnectScreen, MessagesScreen, LeaderboardScreen, ProfileScreen, ClientDashboard, ClientFindTalentScreen, ClientMessagesScreen, ClientProfileScreen, CompletedProjectsScreen
src/components	auth (onboarding modals), connect (search, requests, connections, message thread, team engine), cards, ratings, profile, projects, filters, layout, ui (20 primitives)
src/hooks	useAuth, useUserRole, useProjects / useUserRatings, useNotifications, use-mobile, use-toast
src/integrations/supabase	Typed Supabase client and generated database types
supabase/migrations	8 SQL migrations: tables, enums, RLS policies, storage bucket, trigger

3.10.2 Authentication and Role Routing

`AuthProvider` subscribes to `onAuthStateChange` before reading the existing session, so no auth event is missed during start-up. Network failures are translated into a clear message that points to the `VITE_SUPABASE_URL` setting. The `Index` page waits for both the session and the role to load, redirects anonymous users to `/auth`, and renders either the developer or the client screen set. The flowchart below shows the logic.

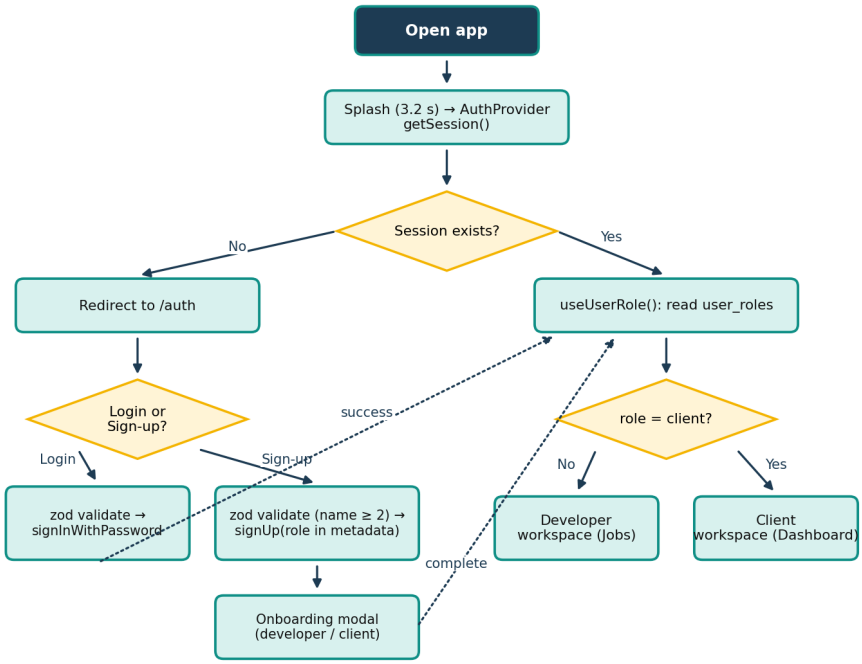


Figure 3.9: Flowchart of authentication and role-based routing

Form inputs are validated with Zod schemas before any network call: e-mail must be well-formed, the password must be non-empty and the full name must have at least two characters.

3.10.3 Onboarding and Profiles

After registration, a three-step modal opens. The **developer** wizard offers three paths (LinkedIn link, résumé upload or manual entry) and finally shows a live preview of the profile card before saving title, bio and skills. The **client** wizard captures company name, website, location, industry, company size, hiring goal and mission statement. The developer profile screen shows avatar, skills, connection count, completed projects, rating summary and six dynamically earned badges; avatars are uploaded to the `avatars` storage bucket under the user's own folder.

3.10.4 Developer Discovery, Connections and Requests

The Connect screen has four tabs: *Find Developers*, *My Connections*, *Team Engine* and *Requests*. Search combines free-text matching on name, type and skills with select-based filters for developer type, skill and a two-level region filter (continent → country). Each

developer card shows the current connection state — *Connect*, *Request Sent* or *Message* — derived from the `connections` table. Incoming requests are loaded for the receiver and refreshed through a realtime subscription.

3.10.5 Real-time Messaging

Each conversation is a `MessageThread` bound to one accepted connection. It loads history ordered by time and subscribes to inserts for that connection only:

Listing 3.3: Realtime subscription in MessageThread

```
const channel = supabase
  .channel(`messages-${connectionId}`)
  .on('postgres_changes',
      { event: 'INSERT', schema: 'public', table: 'messages',
        filter: `connection_id=eq.${connectionId}` },
      (payload) => setMessages(prev => [...prev, payload.new as Message]))
  .subscribe();
return () => { supabase.removeChannel(channel); };
```

The inbox (`MessagesScreen`) lists all accepted connections with the latest message, time and unread count, sorted by recency, and is shared by both roles.

3.10.6 Team Formation Engine

Developers create a *team batch* by giving it a name, a project domain and an objective, and by adding members **only from accepted connections**. Each member can be assigned a project role. As members are added, the engine compares the union of their skills with a recommended skill set for the selected domain and computes a synergy score using Algorithm 1.

Algorithm 1 Synergy Score Calculation for a Team Batch

Input: selected members M, project domain d
R ← DOMAIN_RECOMMENDED_SKILLS[d] ▷ default: Fullstack Web App
S ← ∪ { skills(m) : m ∈ M } ▷ combined stack
covered ← ∅; missing ← ∅
for each skill r ∈ R **do**
  if ∃ s ∈ S : lower(s) = lower(r) or lower(s) contains lower(r) **then** covered ← covered ∪ {r}
  else missing ← missing ∪ {r}
ratio ← |covered| / |R|
bonus ← min(10 × |M|, 30) ▷ team-size bonus
score ← min(round(70 × ratio + bonus), 100); **if** |M| = 0 **then** score ← 0
return score, S, missing

For example, for the *Fullstack Web App* domain (React, TypeScript, Node.js, PostgreSQL, UI/UX Design, AWS), a two-member team covering four of the six skills scores round(70 × 4/6 + 20) = 67 %. Missing skills are shown as *skill gaps to consider*, guiding the developer to invite a complementary connection.

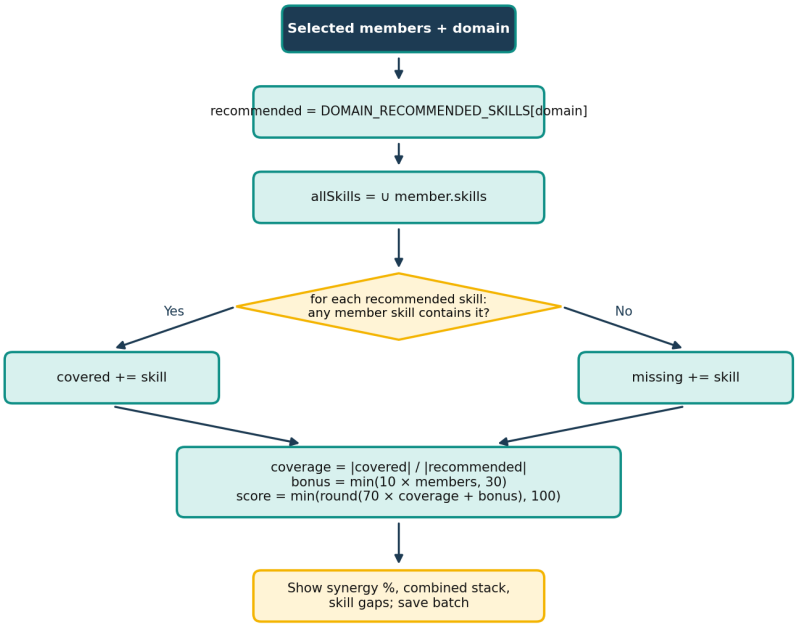


Figure 3.10: Flowchart of the synergy score algorithm

3.10.7 Projects, Ratings and Leaderboard

The `useProjects` hook fetches all projects, the user's own projects and completed projects using embedded joins to profiles and ratings. On the Completed Projects screen, a participant who has not yet rated a project sees *Rate Project* and *Rate Developer/Client* buttons, which open a five-star `RatingDialog`. The completion-gated rating policy shown in the Security Design section makes the final decision, as the following flowchart shows.

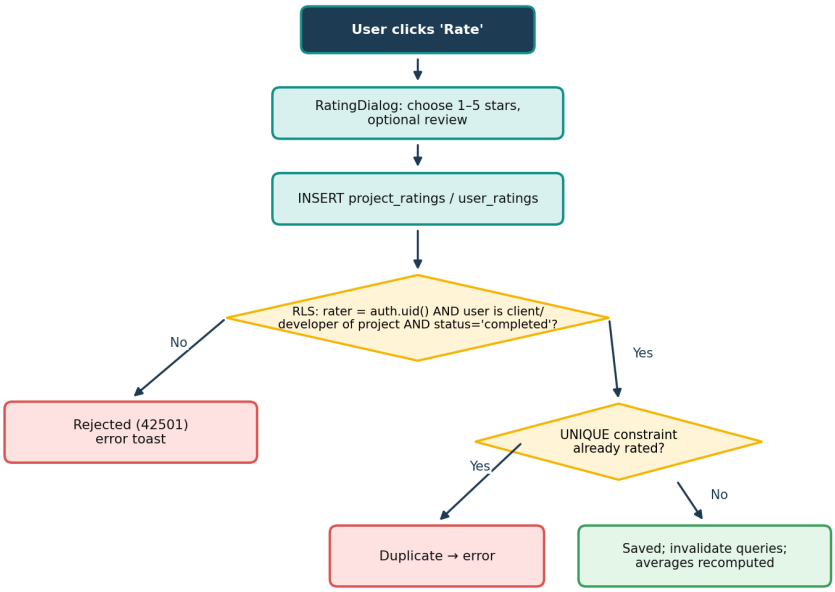


Figure 3.11: Flowchart of rating submission and validation

The leaderboard (Algorithm 2) ranks every non-client profile by points equal to round(average rating × 1000). Profiles without any rating start from a baseline average of 4.5, so new developers are not hidden at the bottom of the list. Gold, silver and bronze badges are

awarded cumulatively at 4500, 4000 and 3500 points. The client dashboard aggregates active projects, total budget, the number of distinct hired developers and the client's own rating.

Algorithm 2 Leaderboard Ranking

```
P ← profiles where role ≠ 'Client'
Rt ← all rows of user_ratings
for each profile p ∈ P do
    Rp ← { r ∈ Rt : r.rated_user_id = p.id }
    avg ← mean(Rp.rating) if Rp ≠ ∅ else 4.5
    points(p) ← round(avg × 1000)
    badges(p) ← {gold if points ≥ 4500} ∪ {silver if ≥ 4000} ∪ {bronze if ≥ 3500}, at least {bronze}
sort P by points in descending order
for i ← 1 to |P| do rank(P[i]) ← i
return ranked list P
```

3.11 Configuration and Deployment

The application reads its backend location from two Vite environment variables. To make a fresh clone runnable even without a local `.env` file, `vite.config.ts` injects public fallback values through Vite's `define` option only when a variable is missing:

Listing 3.4: Environment fallback in vite.config.ts

```
const env = loadEnv(mode, process.cwd(), 'VITE_');
const define = Object.fromEntries(
  Object.entries(FALLBACK_ENV)
    .filter(([key]) => !env[key])
    .map(([key, value]) => [`import.meta.env.${key}`, JSON.stringify(value)]));
```

Only the publishable (anonymous) key is ever shipped to the browser; this is safe because every table is protected by RLS. Deployment consists of three steps:

1. Apply the SQL files in `supabase/migrations` to the Supabase project (tables, policies, trigger and storage bucket) and enable the Google provider if required.
2. Run `npm install` followed by `npm run build`; Vite produces a static `dist/` folder (index.html, one CSS and one JS bundle, and the logo).
3. Upload `dist/` to any static host or CDN and configure it to serve `index.html` for unknown paths so that client-side routes such as `/auth` work on refresh.

Chapter 4

Results and Analysis

4.1 Tools and Technologies

Table 4.1: Tools and technologies used

Category	Tools & technologies
Front-end	React 18.3 · TypeScript 5.8 · Vite 5.4 · Tailwind CSS 3.4 · shadcn/ui (Radix UI) · Lucide icons · React Router 6 · TanStack Query 5 · Zod 3
Back-end (BaaS)	Supabase: PostgreSQL, Row-Level Security, Auth (e-mail, Google OAuth), Realtime, Storage · supabase-js 2.88
Algorithms	Synergy score for team batches (Algorithm 1) · rating-based leaderboard ranking (Algorithm 2)
Development tools	Visual Studio Code · Git & GitHub · Lovable · Supabase Studio · npm
Testing tools	TypeScript compiler · ESLint · Puppeteer + headless Chrome · Lighthouse 12
Communication protocols	HTTPS / REST (PostgREST) · WebSocket (Supabase Realtime) · JWT · OAuth 2.0

4.2 Results

The CollabriX prototype is functional for authentication, onboarding, profiles, developer discovery, connections, real-time messaging, team batch formation, ratings and the leaderboard. The following table contrasts the approach of conventional freelance marketplaces with that of CollabriX, and the screenshots after it show the implemented system.

Table 4.2: Comparative analysis of conventional freelance platforms vs. CollabriX

Criteria	Conventional platforms	CollabriX
Unit of hiring	Individual freelancer	Individual or team batch
Peer discovery	Limited or absent	Search + connect with filters
Team composition insight	Manual judgement	Synergy score and skill gaps
Rating integrity	Platform policy	Enforced by database RLS
Communication	Built-in chat	Realtime chat gated by accepted connection
Access control	Application server code	Row-Level Security in PostgreSQL



Figure 4.1: Splash screen shown while the session is restored

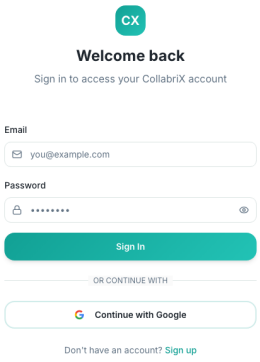


Figure 4.2: Login screen with e-mail/password and Google sign-in

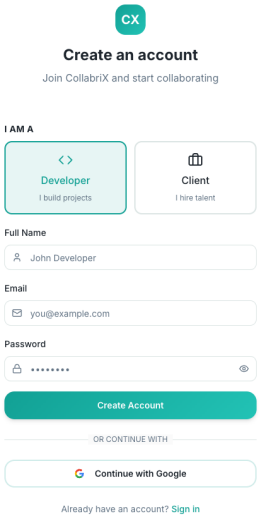


Figure 4.3: Sign-up screen with Developer / Client role selection

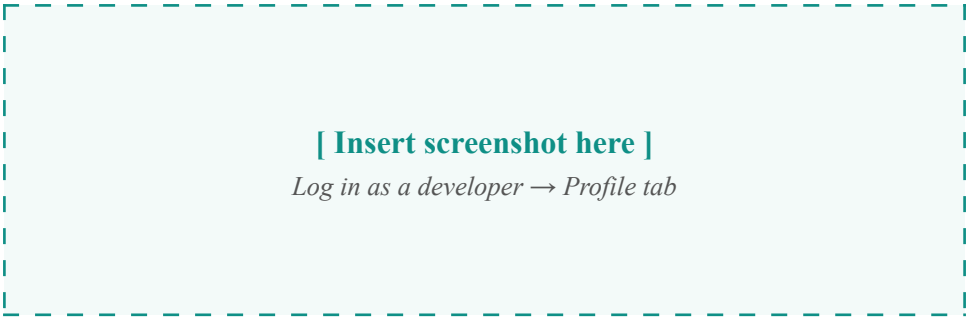


Figure 4.4: Developer profile screen with ratings overview and badges

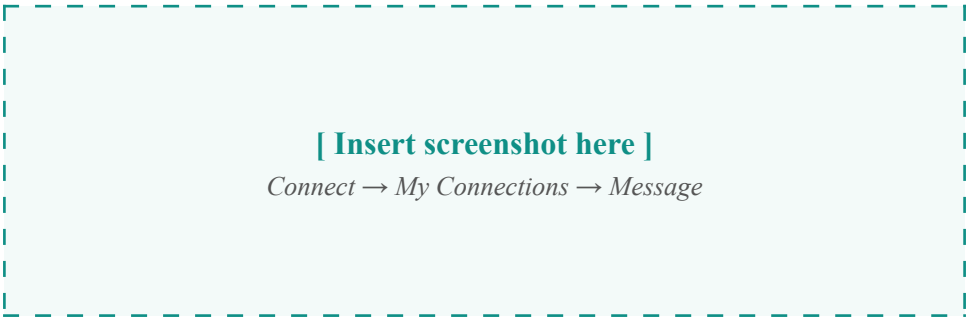


Figure 4.5: Real-time chat between two connected developers

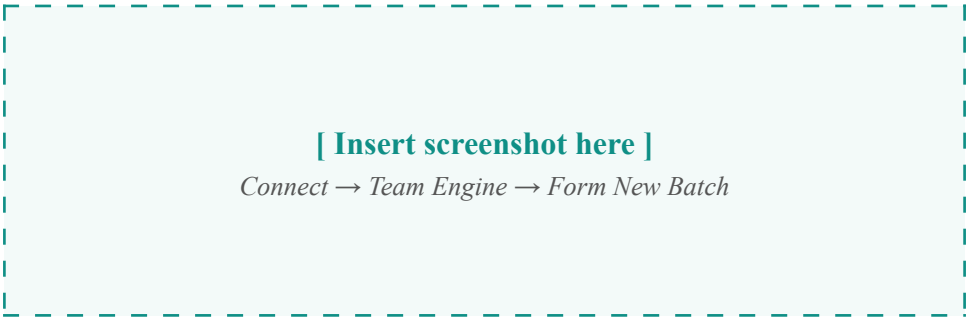


Figure 4.6: Team Formation Engine with synergy score and skill gaps

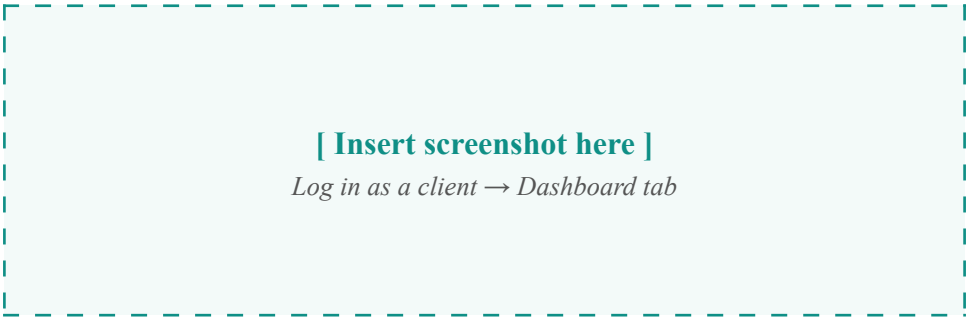


Figure 4.7: Client dashboard with project statistics

4.2.1 Quantitative Results

Testing was carried out at four levels: static analysis, automated user-interface tests, security (RLS) verification and performance auditing. All results in this chapter were obtained by actually running the tools against the project on 6 October 2026.

A. Static analysis and unit-level checks

The project does not yet include a unit test framework such as Vitest; unit-level correctness was therefore checked through the TypeScript compiler, which type-checks every module against the generated database types, and through ESLint.

Table 4.3: Static analysis results

Check	Command	Result
Type checking	npx tsc --noEmit -p tsconfig.app.json	Passed — 0 errors
Linting	npx eslint .	49 findings: 14 errors, 35 warnings
Production build	npx vite build	Succeeded in 1.83 s

The lint findings are dominated by missing dependencies in `useEffect` hooks (20 warnings) and component files that also export constants (15 warnings), neither of which breaks the build. The 10 `no-explicit-any` errors are in error-handling code paths; these are listed as technical debt in Chapter 5.

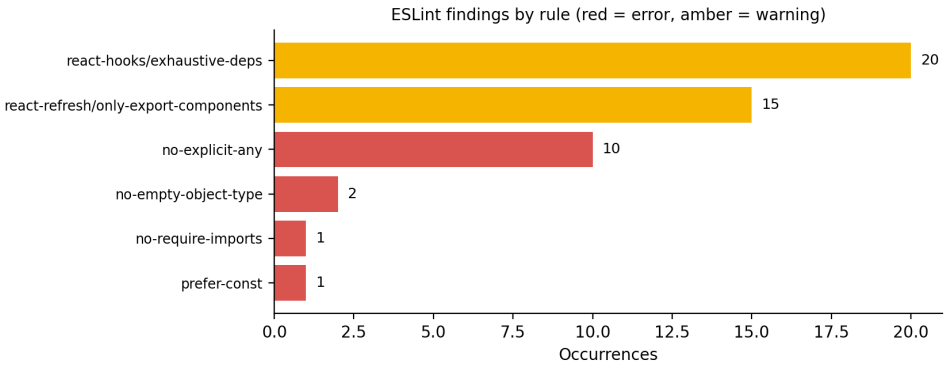


Figure 4.8: ESLint findings grouped by rule

B. Functional test cases (automated UI tests)

A Puppeteer script drove a headless Chrome browser against the development server to validate the authentication and navigation flows. These tests exercise only client-side logic, so they did not create any accounts on the live backend.

Table 4.4: Automated UI test cases and results

ID	Test case	Expected result	Actual result	Status
TC-01	Open "/" without a session	Redirect to /auth	Redirected to /auth	Pass
TC-02	Log in with malformed e-mail	Submission blocked	Browser validation: "Please include an '@'…"	Pass
TC-03	Log in with empty password	Toast "Please enter a password"	Toast shown	Pass
TC-04	Toggle password visibility	Type password → text	password → text	Pass
TC-05	Switch to sign-up mode	Heading and Full Name field	"Create an account", field shown	Pass
TC-06	Select Client role card	Card highlighted	Highlighted	Pass

ID	Test case	Expected result	Actual result	Status
TC-07	Sign up with 1-letter name	Toast “Name must be at least 2 characters”	Toast shown	Pass
TC-08	Open an unknown route	404 page	“Oops! Page not found”	Pass
TC-09	Login page at 375 px width	No horizontal overflow	scrollWidth 375 = viewport 375	Pass

All nine automated test cases passed. In TC-02 the request was stopped by the browser's built-in HTML5 e-mail validation before the application's own Zod check, which is the desired outcome.

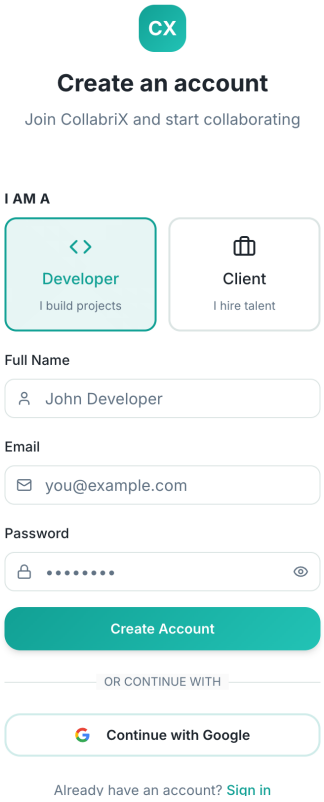


Figure 4.9: Sign-up screen rendered at mobile width (TC-09)

C. Manual test cases for authenticated features

Table 4.5: Manual test cases for authenticated features

ID	Scenario	Expected result
MT-01	Developer A sends a connection request to B	B sees the request with a badge; A sees “Request Sent”
MT-02	B accepts the request	Both see each other under My Connections; Message enabled
MT-03	A sends a message	Message appears in B's open thread without refresh
MT-04	Create a batch with two connections	Synergy score and skill gaps displayed; batch listed
MT-05	Client opens Find Talent with region filter	Only developers from the selected country/ continent listed

ID	Scenario	Expected result
MT-06	Participant rates a completed project twice	Second attempt rejected by the unique constraint

These scenarios require two real user accounts on the live backend, so they were not automated; they serve as the acceptance checklist to be run with two test accounts before the final demonstration.

D. Security testing (Row-Level Security validation)

To validate NFR-1, an anonymous Supabase client (using only the public key that ships in the browser) queried each table and attempted unauthenticated inserts against the live project. Only row counts were recorded.

Table 4.6: RLS validation with an anonymous client

Table / operation	Expected	Observed	Status
SELECT profiles	Readable (public directory)	9 rows visible	Pass
SELECT projects	Readable (public listings)	0 rows (none posted yet)	Pass
SELECT connections	Not readable	0 rows	Pass
SELECT messages	Not readable	0 rows	Pass
SELECT user_roles	Not readable	0 rows	Pass
SELECT ratings tables	Readable but empty	0 rows	Pass
INSERT projects (anon)	Rejected	42501: violates row-level security policy	Pass
INSERT messages (anon)	Rejected	42501: violates row-level security policy	Pass

E. Performance testing

The production build was served locally with `vite preview` and audited with Lighthouse 12 using its default mobile emulation (simulated slow 4G and CPU throttling).

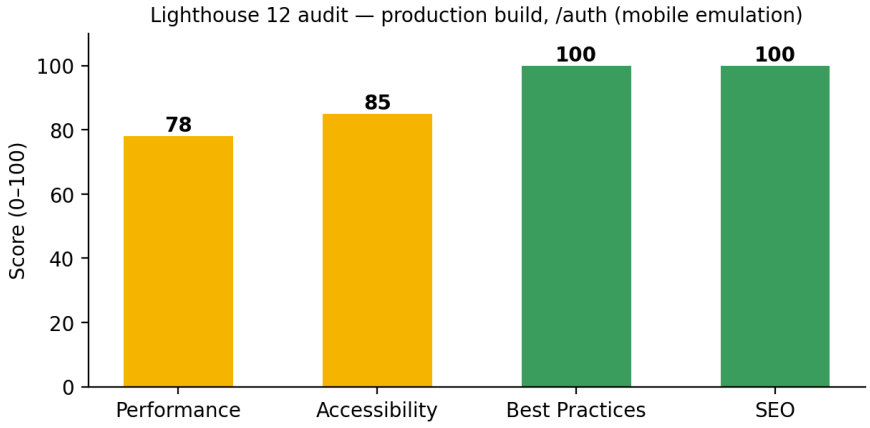


Figure 4.10: Lighthouse category scores for the production build

Table 4.7: Lighthouse metrics and bundle sizes

Metric	Value	Remark
First Contentful Paint	3.3 s	Throttled mobile profile
Largest Contentful Paint	4.2 s	Dominated by the 827 kB JS bundle
Total Blocking Time	10 ms	Main thread stays responsive
Cumulative Layout Shift	0	No layout shifts
Speed Index	4.0 s	—
JavaScript bundle	826.8 kB (231.3 kB gzip)	Single chunk; code-splitting recommended
CSS bundle	62.5 kB (10.4 kB gzip)	Tailwind purged output
Logo image	163.8 kB	Candidate for compression / WebP

The main performance bottleneck is the single 827 kB JavaScript chunk, which Vite itself flags with a size warning. Splitting the client and developer workspaces into lazily loaded chunks and compressing the logo are expected to raise the Performance score above 90.

F. Outcome discussion

The security model was compared against an external reference, the OWASP Top 10 [16]: broken access control (A01) is mitigated by RLS, injection (A03) by parameterised PostgREST queries, and identification failures (A07) by Supabase Auth. Testing also revealed defects that are documented in the Limitations section of Chapter 5 — for example, the Jobs screen offers *Beginner/Intermediate/Advanced* filters while the database stores *Easy/Medium/Hard*, so that filter currently returns no results.

Chapter 5

Conclusion and Future Works

5.1 Conclusion

CollabriX demonstrates that a freelance platform can be designed around teams rather than individuals. The prototype delivers two complete role-based workspaces, guided onboarding, developer discovery with faceted filters, a request-and-accept connection workflow, real-time messaging, a team formation engine with an explainable synergy score, and a reputation system whose integrity is enforced by the database. The entire authorisation model lives in PostgreSQL Row-Level Security policies, which our tests confirmed: no private data is readable without authentication and unauthenticated writes are rejected.

From an engineering perspective, the project gave the team practical experience with a modern TypeScript front-end stack, relational schema design, SQL-based security, realtime systems, agile delivery in short sprints and systematic testing.

Table 5.1: Project goals versus accomplishments

Objective	Accomplishment	Status
Role-based responsive SPA	Developer (5 tabs) and client (4 tabs) workspaces; works at 375 px	Achieved
Peer discovery and connections	Search, type/skill/region filters, request–accept workflow	Achieved
Real-time messaging	WebSocket subscriptions per connection; shared inbox	Achieved
Team batches with synergy score	Built from accepted connections with roles and gaps; stored in browser	Partially achieved
Completion-gated reputation	RLS-enforced ratings, leaderboard and badges; rating UI on completed projects	Achieved
Database-level security	RLS on all 7 tables and storage; validated by tests	Achieved
Evaluation and documentation	UI, RLS, static and performance tests; limitations documented	Achieved

5.1.1 Lessons Learned

- **Put the rules in the database first.** Writing RLS policies before the screens forced us to decide early who may read and write each table, and it made the front end simpler because it never has to re-check permissions.
- **Realtime needs clean-up.** Every Supabase channel opened in a React effect must be removed when the component unmounts; otherwise duplicate subscriptions and repeated notifications appear.

- **Demo data hides gaps.** Sample connections and default ratings made early demos look complete, but they also concealed missing workflows. We now prefer honest empty states.
- **Keep the data model aligned with the UI.** The complexity-filter mismatch showed that enumerated values should be defined once and shared between SQL and TypeScript.
- **Generated code still needs review.** AI-assisted scaffolding (Lovable) sped up UI work, but generated files need the same review as hand-written code; for example, one onboarding modal was rendered twice on the client profile screen.

5.2 Limitations

Table 5.2: Known limitations of the current prototype

Area	Limitation
Project lifecycle	Clients cannot yet post projects, assign a developer or mark a project complete from the UI; these steps require database access, so the rating flow is only reachable with seeded data.
Messaging	The `read` flag is never set, and the RLS policy only lets the sender update a message, so unread counts do not decrease.
Team batches	Batches are stored in browser localStorage, are not shared with team members, and include a demo batch.
Demo data	When a user has no connections, sample demo connections and a default 4.8★ rating are displayed; unrated developers appear on the leaderboard with a baseline average of 4.5.
Filters	Jobs complexity filter values (Beginner/Intermediate/Advanced) do not match stored values (Easy/Medium/Hard).
Onboarding	LinkedIn and résumé options do not extract data; client company fields reuse developer profile columns.
Code quality	14 ESLint errors and 35 warnings; no automated unit test suite yet.
Performance	Single 827 kB JavaScript bundle; Lighthouse Performance 78.

5.3 Future Prospectus

1. **Complete project lifecycle:** project posting form, proposals from individuals and team batches, assignment, milestones and completion, so that ratings flow naturally.
2. **Server-side team batches:** move batches into a `teams` table with RLS, member invitations and a shared team chat.
3. **Read receipts:** a policy allowing receivers to mark messages read, plus typing indicators using Realtime Presence.
4. **Smarter team suggestions:** recommend connections that fill skill gaps, weighting ratings and current workload, following [5] and [11].
5. **Multi-dimensional, double-blind ratings:** separate scores for quality, communication and timeliness, revealed only after both parties submit, to counter reputation inflation [8].

6. **Payments and escrow:** milestone-based payments through a payment gateway such as Razorpay or Stripe.
7. **Quality and performance:** add Vitest and Playwright test suites, fix lint findings, code-split the bundle and compress images.

5.3.1 Possible Integration with Other Tools

- **GitHub** — link repositories to profiles and show contribution activity as verifiable evidence of skills.
- **Google Calendar / Meet** — schedule and join client meetings directly from a conversation.
- **Slack or Discord webhooks** — mirror team notifications for users who prefer those tools.
- **LinkedIn** — official OAuth-based profile import in place of the current link field.

Bibliography

[1] J. J. Horton, "Online labor markets," in *Proc. 6th Int. Workshop on Internet and Network Economics (WINE)*, LNCS, vol. 6484, Springer, 2010, pp. 515–522.

[2] A. Kittur, J. V. Nickerson, M. Bernstein, E. Gerber, A. Shaw, J. Zimmerman, M. Lease, and J. Horton, "The future of crowd work," in *Proc. ACM Conf. Computer Supported Cooperative Work (CSCW)*, 2013, pp. 1301–1318.

[3] P. Resnick, K. Kuwabara, R. Zeckhauser, and E. Friedman, "Reputation systems," *Communications of the ACM*, vol. 43, no. 12, pp. 45–48, 2000.

[4] A. Jøsang, R. Ismail, and C. Boyd, "A survey of trust and reputation systems for online service provision," *Decision Support Systems*, vol. 43, no. 2, pp. 618–644, 2007.

[5] T. Lappas, K. Liu, and E. Terzi, "Finding a team of experts in social networks," in *Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2009, pp. 467–476.

[6] D. Retelny, S. Robaszkiewicz, A. To, W. S. Lasecki, J. Patel, N. Rahmati, T. Doshi, M. Valentine, and M. S. Bernstein, "Expert crowdsourcing with flash teams," in *Proc. 27th ACM Symp. User Interface Software and Technology (UIST)*, 2014, pp. 75–85.

[7] M. A. Valentine, D. Retelny, A. To, N. Rahmati, T. Doshi, and M. S. Bernstein, "Flash organizations: Crowdsourcing complex work by structuring crowds as organizations," in *Proc. CHI Conf. Human Factors in Computing Systems*, 2017, pp. 3523–3537.

[8] A. Filippas, J. J. Horton, and J. Golden, "Reputation inflation," *Marketing Science*, vol. 41, no. 4, pp. 733–745, 2022.

[9] M. Kokkodis and P. G. Ipeirotis, "Reputation transferability in online labor markets," *Management Science*, vol. 62, no. 6, pp. 1687–1706, 2016.

[10] A. J. Wood, M. Graham, V. Lehdonvirta, and I. Hjorth, "Good gig, bad gig: Autonomy and algorithmic control in the global gig economy," *Work, Employment and Society*, vol. 33, no. 1, pp. 56–75, 2019.

[11] A. Anagnostopoulos, L. Becchetti, C. Castillo, A. Gionis, and S. Leonardi, "Online team formation in social networks," in *Proc. 21st Int. Conf. World Wide Web (WWW)*, 2012, pp. 839–848.

[12] C. Dellarocas, "The digitization of word of mouth: Promise and challenges of online feedback mechanisms," *Management Science*, vol. 49, no. 10, pp. 1407–1424, 2003.

[13] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, IETF, Dec. 2011.

[14] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015.

[15] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.

[16] OWASP Foundation, "OWASP Top 10:2021." [Online]. Available: https://owasp.org/Top10/

[17] Supabase, "Row Level Security," Supabase Documentation. [Online]. Available: https://supabase.com/docs/guides/database/postgres/row-level-security

[18] Meta Open Source, "React — The library for web and native user interfaces." [Online]. Available: https://react.dev

[19] Vite, "Vite — Next generation frontend tooling." [Online]. Available: https://vitejs.dev

[20] TanStack, "TanStack Query documentation." [Online]. Available: https://tanstack.com/query

[21] Tailwind Labs, "Tailwind CSS documentation." [Online]. Available: https://tailwindcss.com/docs

[22] The PostgreSQL Global Development Group, "Row Security Policies," PostgreSQL Documentation. [Online]. Available: https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[23] Google, "Lighthouse overview," Chrome for Developers. [Online]. Available: https://developer.chrome.com/docs/lighthouse/overview

[24] K. Beck *et al.*, "Manifesto for Agile Software Development," 2001. [Online]. Available: https://agilemanifesto.org

[25] IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998, 1998.